

```
import pandas as pd

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
X, y = make_classification(
    n_samples=2000,
    n_features=6,
    n_informative=5,
    n_redundant=1,
    n_classes=2,
    class_sep=2,
    random_state=42
)

columns = [
    "Length",
    "Width",
    "Weight",
    "Thickness",
    "SurfaceScore",
    "ColorVariation"
]
```

```
df = pd.DataFrame(X, columns=columns)
df["Quality"] = y

print("\nFirst 5 Rows")
print(df.head())
```

```
First 5 Rows
```

|   | Length    | Width     | Weight    | Thickness | SurfaceScore | ColorVariation \ |
|---|-----------|-----------|-----------|-----------|--------------|------------------|
| 0 | -4.367467 | 4.244811  | 3.022927  | 2.756156  | 5.274261     | 1.405176         |
| 1 | -3.911271 | 2.757383  | 1.397813  | 2.496144  | 3.961175     | 1.575626         |
| 2 | 1.529241  | 1.270890  | 0.693553  | 1.342251  | 1.400030     | -2.699872        |
| 3 | 2.804603  | -1.139249 | -1.161813 | 2.785905  | 2.942281     | -1.365394        |
| 4 | 1.150223  | -0.238191 | 1.579840  | -1.933873 | -1.299633    | -0.563968        |

  

|   | Quality |
|---|---------|
| 0 | 1       |
| 1 | 0       |
| 2 | 0       |
| 3 | 0       |
| 4 | 1       |

```
X = df.drop("Quality", axis=1)
y = df["Quality"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print("\n=====")
print("PERCEPTRON MODEL")
print("=====")
```

```

perceptron = Perceptron(random_state=42)

perceptron.fit(X_train, y_train)

pred1 = perceptron.predict(X_test)

print("Accuracy :", round(accuracy_score(y_test, pred1) * 100, 2), "%")

print("\nClassification Report")
print(classification_report(y_test, pred1))

print("Confusion Matrix")
print(confusion_matrix(y_test, pred1))

```

```

=====
PERCEPTRON MODEL
=====
Accuracy : 91.5 %

Classification Report
      precision    recall  f1-score   support

     0       0.93      0.90      0.91       202
     1       0.90      0.93      0.92       198

   accuracy       0.92
  macro avg       0.92
weighted avg       0.92

Confusion Matrix
[[181  21]
 [ 13 185]]

```

```

print("\n=====")
print("MLP MODEL")
print("=====")

mlp = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation="relu",
    max_iter=1000,
    random_state=42
)

mlp.fit(X_train, y_train)

pred2 = mlp.predict(X_test)

print("Accuracy :", round(accuracy_score(y_test, pred2) * 100, 2), "%")

print("\nClassification Report")
print(classification_report(y_test, pred2))

print("Confusion Matrix")
print(confusion_matrix(y_test, pred2))

# =====
# MODEL COMPARISON
# =====

print("\n=====")
print("MODEL COMPARISON")
print("=====")

print("Perceptron Accuracy :", round(accuracy_score(y_test, pred1) * 100, 2), "%")
print("MLP Accuracy          :", round(accuracy_score(y_test, pred2) * 100, 2), "%")

```

```

=====
MLP MODEL
=====
Accuracy : 99.5 %

Classification Report
      precision    recall  f1-score   support

     0       0.99      1.00      1.00       202
     1       1.00      0.99      0.99       198

```

```

accuracy          0.99    400
macro avg         1.00    0.99    0.99    400
weighted avg      1.00    0.99    0.99    400

```

Confusion Matrix

```

[[202  0]
 [ 2 196]]

```

```

=====
MODEL COMPARISON
=====

```

```

Perceptron Accuracy : 91.5 %
MLP Accuracy       : 99.5 %

```

```

print("\nEnter Product Details")

length = float(input("Length : "))
width = float(input("Width : "))
weight = float(input("Weight : "))
thickness = float(input("Thickness : "))
surface = float(input("Surface Score : "))
color = float(input("Color Variation : "))

sample = [[length, width, weight, thickness, surface, color]]

sample = scaler.transform(sample)

prediction = mlp.predict(sample)

print("\nPrediction")

if prediction[0] == 1:
    print("GOOD PRODUCT")
else:
    print("DEFECTIVE PRODUCT")

```

```

Enter Product Details
Length : 11
Width : 2
Weight : 21
Thickness : 12
Surface Score : 11
Color Variation : 23

```

```

Prediction
GOOD PRODUCT

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(

```